

 Extract Interface

تعریف ساده

وقتی چند کلاس رفتار مشترک دارن یا وقتی می‌خوای فقط بخشی از متدهای یه کلاس رو در دسترس بساز و اون متدهای مشترک رو توش تعریف کن (رابط) **Interface** بقیه بذاری، یه

 Extract Class و Extract Subclass تفاوت با

تکنیک	رابطه	کاربرد
Extract Class	Has-A (ترکیب)	جدا کردن مسئولیت‌ها
Extract Subclass	Is-A (ارث‌بری)	حالت‌های خاص یه مفهوم
Extract Interface	Can-Do (توانایی)	رفتار مشترک بین کلاس‌های نامرتب

 مثال ساده

قبل از ریفاکتور:

```
csharp
public class ReportGenerator
{
    public void GeneratePDF() { /* ... */ }
    public void GenerateExcel() { /* ... */ }
    public void SendEmail(string to) { /* ... */ }
    public void SaveToDatabase() { /* ... */ }
}

public class InvoiceProcessor
{
    public void CalculateInvoice() { /* ... */ }
    public void GeneratePDF() { /* ... */ }
    public void SendEmail(string to) { /* ... */ }
    public void Print() { /* ... */ }
}
```

دو کلاس مختلف، ولی رفتارهای مشترک دارن: `GeneratePDF` و `SendEmail`. الان نمیشه با اینها یه‌جور برخورد کرد.

Extract Interface: بعد از

```
csharp
// استخراج رفتار مشترک
public interface IExportable
{

```

```

void GeneratePDF();
void SendEmail(string to);
}

public class ReportGenerator : IExportable
{
    public void GeneratePDF() { /* ... */ }
    public void GenerateExcel() { /* ... */ }
    public void SendEmail(string to) { /* ... */ }
    public void SaveToDatabase() { /* ... */ }
}

public class InvoiceProcessor : IExportable
{
    public void CalculateInvoice() { /* ... */ }
    public void GeneratePDF() { /* ... */ }
    public void SendEmail(string to) { /* ... */ }
    public void Print() { /* ... */ }
}

// حالا میتونی با هر دو به یه شکل برخورد کنی:
List<IExportable> exportables = new() { new ReportGenerator(), new InvoiceProcessor() };
foreach (var item in exportables)
{
    item.GeneratePDF();
    item.SendEmail("admin@site.com");
}

```

مزایا

مزیت	توضیح
انعطاف‌پذیری	کلاس‌های نامرتب می‌تونن رفتار مشترک داشته باشن
Dependency Inversion	وابسته میشی Interface به جای وابستگی به کلاس مشخص، به
تست آسون‌تر	کنی Mock رو Interface می‌تونی
پنهان‌سازی	هستن در دسترس بقیه قرار Interface فقط متدهایی که توی می‌گیرن
Large Class جلوگیری از	کلاس رو مجبور می‌کنی به فکر کردن درباره "چه رفتاری رو در معرض دید بذارم"

تمرین

ریفکتور کن Extract Interface اینا رو با

```

csharp

public class UserService
{

```

```

    public void RegisterUser(string name, string email) { /* ... */ }
    public void DeleteUser(int id) { /* ... */ }
    public void LogActivity(string message) { /* ... */ }
    public void SendNotification(string to, string text) { /* ... */ }
    public string ExportData(string format) { /* ... */ }
}

public class ProductService
{
    public void AddProduct(string name, decimal price) { /* ... */ }
    public void RemoveProduct(int id) { /* ... */ }
    public void LogActivity(string message) { /* ... */ }
    public void SendNotification(string to, string text) { /* ... */ }
    public string ExportData(string format) { /* ... */ }
}

public class ReportService
{
    public void GenerateReport() { /* ... */ }
    public void LogActivity(string message) { /* ... */ }
    public void SendNotification(string to, string text) { /* ... */ }
    public string ExportData(string format) { /* ... */ }
    public void ScheduleReport(DateTime date) { /* ... */ }
}

```

ها رو بسازی Interface راهنمایی: سه تا رفتار مشترک بین این کلاس ها هست. می تونی این

- ILogger → LogActivity
- INotifier → SendNotification
- IExportable → ExportData

👉! بفرست تا بررسی کنیم